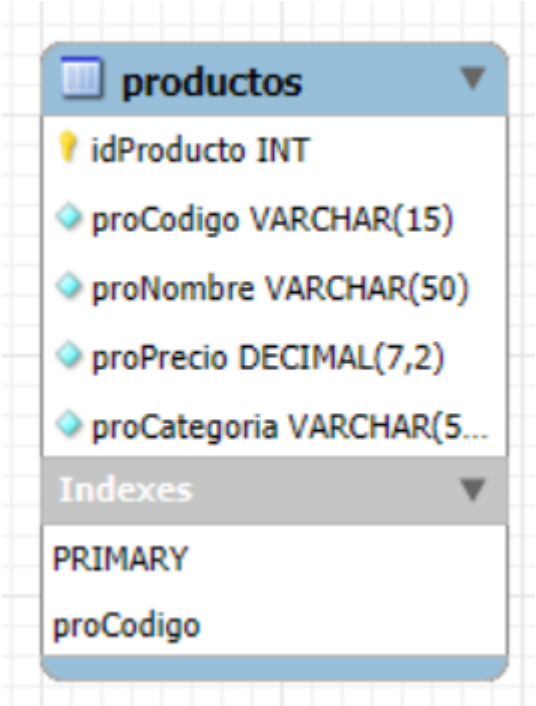
PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
MATERIAL DE APOYO
CONEXIÓN A BASES DE DATOS DESDE PYTHON

Introducción

El presente material de apoyo contiene todo el proceso de conexión a una base de datos **mysql** desde **Python** y el proceso de consultas de un **CRUD** a una base de datos.

La base de datos a utilizar contiene una sola tabla así:



Para conectarnos a bases de datos mysql necesitamos tener instalado mysql y el controlador que permita la conexión entre Python y mysql.

Controladores existentes:

- Mysql-connector
- MYsqlDb
- Pymysql

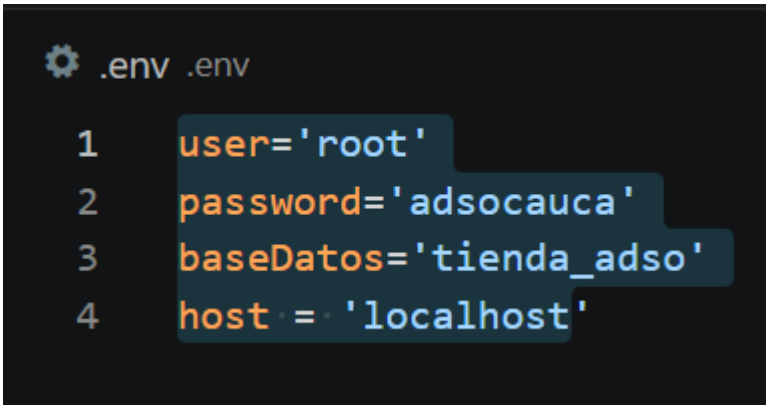
Para el presente material vamos a utilizar el controlador llamado **pymysql.** Para instalar ingresar el siguiente comando en la consola:

pip install pymysql

Después de tener ya instalado nuestro controlador, proceder a crear una base de datos en mysql con el nombre de ***tienda_adso***

Código de la Solución

Código archivo .**env** con variables de entorno



Código archivo app.py



```
Tabnine | Edit | Test | Explain | Document
def agregar():
    try:
    #creamos una tupla con el producto
    # produco(codigo, nombre, precio, categoria))
      producto = ("31","Botas",32000,"Calzado")
      consulta = "insert into productos values(null, %s,%s,%s,%s)"
      cursor.execute(consulta,producto)
      miConexion.commit()
      if(cursor.rowcount==1):
          print("Producto agregado correctamente")

    except miConexion.Error as e:
        miConexion.rollback()
        print(str(e))
```

```
Tabnine | Edit | Test | Explain | Document
def listar():
    try:
        consulta = "select * from productos"
        cursor.execute(consulta)
        productos = cursor.fetchall()
        for producto in productos:
            print(producto)
            print(f"Id: {producto[0]}")
            print(f"Código: {producto[1]}")
            print(f"Nombre: {producto[2]}")
            print(f"Precio: {producto[3]}")
            print(f"Categoría: {producto[4]}")

    except miConexion.Error as e:
         print(str(e))
```

```
Tabnine | Edit | Test | Explain | Document
def consultarPorCodigo():
    try:
        codigo = input("Ingrese código de producto a consultar: ")
        codigoAConsultar=(codigo,)
        consulta= "select * from productos where proCodigo=%s"
        cursor.execute(consulta,codigoAConsultar)
        producto = cursor.fetchone()
        if producto:
            print(producto)
        else:
            print("No existe producto con ese código")

    except miConexion.Error as e:
            print(str(e))
```

```
Tabnine | Edit | Test | Explain | Document
def actualizar():
    try:
        #datosActualizar=("nuevoNombre",idProducto)
        datosActualizar=("Televisor",1)
        consulta = "update productos set proNombre=%s where idProducto=%s"
        cursor.execute(consulta, datosActualizar)
        miConexion.commit()
        if cursor.rowcount==1:
            print("Producto Actualizado")
        else:
            print("No existe producto con ese ID")
    except miConexion.Error as e:
        miConexion.rollback()
        print(str(e))
```

```
Tabnine | Edit | Test | Explain | Document
86  def eliminar():
87      try:
88          # productoEliminar=(idProducto,)
89          productoEliminar=(3,)
90          consulta = "delete from productos where idProducto=%s"
91          cursor.execute(consulta, productoEliminar)
92          miConexion.commit()
93          if cursor.rowcount==1:
94              print("Producto Eliminado")
95          else:
96              print("No existe producto con ese ID")
97      except miConexion.Error as e:
98          miConexion.rollback()
99          print(str(e))
```

```
Tabnine | Edit | Test | Explain | Document
101 def agregarVarios():
102     try:
103         #hacer una lista con varios productos
104         productos= [ ("31","Lavadora",32000,"Electrodomestico"),
105                      ("32","Tennis",32000,"Calzado"),
106                      ("33","Camisa",10000,"Ropa"),
107                      ("34","Chaqueta",99000,"Ropa"),
108                      ("35","Chanclas",11000,"Calzado"),
109                      ("36","Nevera",32000,"Electrodomestico")]
110         consulta = "insert into productos values(null, %s,%s,%s,%s)"
111         cursor.executemany(consulta,productos)
112         miConexion.commit()
113         if(cursor.rowcount==len(productos)):
114             print("Productos agregados correctamente")
115
116     except miConexion.Error as e:
117         miConexion.rollback()
118         print(str(e))
119
```

Código al finalizar el archivo para llamar a cada función

```
133  agregar()
134  #listar()
135  #consultarPorCodigo()
136  #actualizar()
137  #eliminar()
138  #agregarVarios()
```

Resultado de la ejecución del método agregar

```
C:\Users\AdminSena\Documents\SENA2026\GRUPOS\ADSO
\Proyectos\mysql-crud>py app.py
Producto agregado correctamente
```

Resultado ejecución listar

```
C:\Users\AdminSena\Documents\SENA2026\GRUPOS\ADSO 3
\Proyectos\mysql-crud>py app.py
(1, '31', 'Botas', Decimal('32000.00'), 'Calzado')
Id: 1
Código: 31
Nombre: Botas
Precio: 32000.00
Categoría: Calzado
```

Resultado ejecución consultarPorCódigo

```
C:\Users\AdminSena\Documents\SENA2026\GRUPOS\ADSO 3
\Proyectos\mysql-crud>py app.py
Ingrese código de producto a consultar: 31
(1, '31', 'Botas', Decimal('32000.00'), 'Calzado')
```

Resultado eliminar producto de id 1

```
C:\Users\AdminSena\Documents\SENA2026\GRUPOS\ADSO
\Proyectos\mysql-crud>py app.py
Producto Eliminado
```

Resultado agregar varios y se lla a listar para ver resultado.

```
C:\Users\AdminSena\Documents\SENA2026\GRUPOS\ADSO
\Proyectos\mysql-crud>py app.py
Productos agregados correctamente
```

Uso de Commit:

Cada vez que usamos la función **execute()** comienza una nueva transacción a la base de datos que debe confirmarse con **commit()** o bien deshacerse con **rollback().**

¿Qué es un Cursor?

El objeto cursor en Python es una herramienta poderosa que nos permite interactuar con bases de datos. Gracias a él, podemos ejecutar consultas, insertar datos, actualizar registros y más.

¿Qué es conexión rollback?

El comando de la conexión **rollback** forma parte de los módulos del trabajo con base de datos de **SQL**. Es un módulo que pertenece a la librería sql3 de Python, que controla el paso de conexión entre el programa de código y las tablas o bases de datos.

Por tanto, al hablar de qué es conexión rollback podemos decir que ***se encarga de regresar al lugar del desarrollo del programa donde se realizó el último* commit *de guardado***.

Por ejemplo, si se realizan varias operaciones dentro del programa, estas se guardan en un espacio que no ha sido almacenado aún y, si todo sale mal o hay muchos problemas con estos cambios, el comando connection.rollback () regresa al inicio del programa, antes de la ejecución de estos cambios. De esta manera, no se deja ninguna operación a medias.

Referencias:

- Tutorial Python: https://www.w3schools.com/python
- Documentación oficial Python: https://docs.python.org/es/3.9/library/io.html?highlight=working%20streams
- https://www.tutorialspoint.com/python3/python_database_access.htm
- https://parzibyte.me/blog/2018/09/18/python-3-mysql-crud-ejemplos-conexion/

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	20-10-2026